airfoilLaminarFoam

Custom Standalone Laminar SIMPLE Solver

2-D Incompressible Laminar Flow Over a NACA 0012 Airfoil

OpenFOAM Programming Course — Final Mini-Project

Submitted: 30 March 2026 | OpenFOAM v13 (build 13-cde978a97c93)

Solver: airfoilLaminarFoam | Mesh: airFoil2D | Re ≈ 2×10^6

U_∞ = 30 m/s • $\nu = 1.5\times10^{-5}$ m^2/s • 10 SIMPLE iterations • 0.75 s CPU time

Submitted by: Kanishka Awasthi
Email: kanishkaawasthi62@gmail.com

1. Introduction & Project Objectives

This report documents the design, implementation, and execution of **airfoilLaminarFoam** — a custom standalone laminar SIMPLE solver built from scratch within the OpenFOAM v13 framework, without inheriting from any existing solver module.

Project Goals

- Implement a custom steady-state SIMPLE solver using raw fvc/fvm operators — no module inheritance
- Author a bespoke createFields.H reading U and p, registering viscosity and momentum transport models for forceCoeffs
- Configure case from scratch: boundary conditions, physical properties, numerical schemes, linear solver settings
- Run the solver, observe convergence behaviour, identify limitations introduced by solver simplifications
- Post-process results in ParaView; compute Cl/Cd via the forceCoeffs functionObject

Test Case Summary

- Geometry: NACA 0012 (airFoil2D tutorial polyMesh base), AoA = 0°
- Flow: Incompressible laminar, U_∞ = 30 m/s, $\nu = 1.5\times10^{-5}$ m²/s, **$Re \approx 2\times10^{6}$**
- Platform: OpenFOAM v13 (build 13-cde978a97c93) on LAPTOP-SJJBL969, 1 processor

2. OpenFOAM v13 Architecture

OpenFOAM v13 introduces a modular architecture where foamRun dynamically loads a solver module. This project deliberately bypasses that — constructing a fully standalone application calling OpenFOAM library routines directly via fvc::/fvm:: operators.

C++ Class Hierarchy Utilised

- **fvMesh** — mesh topology, cell volumes, face areas
- **volScalarField / volVectorField** — p and U fields, registered with mesh object registry
- **surfaceScalarField phi** — face flux, initialised from fvc::interpolate(U) & Sf()
- **simpleControl** — owns the SIMPLE loop convergence check, reads residualControl
- **incompressible::momentumTransportModel** — registered in mesh registry for forceCoeffs devTau lookup
- **fvVectorMatrix / fvScalarMatrix** — discrete momentum and pressure-correction equations

3. Custom Solver Design — Header Files

airfoilLaminarFoam.H — Include Manifest

- #include "simpleControl.H" — SIMPLE loop convergence control
- #include "constrainHbyA.H" / "constrainPressure.H" — Rhie-Chow correction
- #include "incompressibleMomentumTransportModel.H" — OF v13 API (replaces singlePhaseTransportModel)
- #include "viscosityModel.H" — viscosityModel::New factory; nu() accessor interface

createFields.H — Field Initialisation

- Constructs volScalarField p and volVectorField U with IOobject::MUST_READ | AUTO_WRITE
- Reads nu directly from constant/physicalProperties dictionary — nu = 1.5×10^{-5} m^2/s
- Initialises face flux phi = fvc::interpolate(U) & Sf(); registers incompressible::momentumTransportModel
- Prints diagnostics: initial p, U, nu fields and pRefCell/pRefValue to terminal at startup

4. SIMPLE Algorithm — airfoilLaminarFoam.C

The main file implements the four-step SIMPLE cycle within while(simple.loop(runTime)):

Step 1 — Momentum Equation

- Ueqn = fvm::div(phi, U) - fvm::laplacian(nu, U); Ueqn.relax();
- Momentum predictor disabled (momentumPredictor = no in fvSolution)

Step 2 — Extract A and H; Form HbyA

- A_inv = 1/Ueqn.A(); HbyA = constrainHbyA(A_inv * Ueqn.H(), U, p)

Step 3 — Pressure Poisson Equation

- pEqn = fvm::laplacian(A_inv, p) == fvc::div(phiHbyA); pEqn.setReference(pRefCell, 0)
- Solved via GAMG + GaussSeidel smoother; αp = 0.1 under-relaxation applied via p.relax()

Step 4 — Velocity Correction

- U = HbyA - A_inv * fvc::grad(p); phi = phiHbyA - pEqn.flux(); U.correctBoundaryConditions()

```
fvVectorMatrix Ueqn
(
       fvm::div(phi, U)           // convective term (implicit
upwind)
  - fvm::laplacian(nu, U)  // diffusion term  (implicit central)
);
Ueqn.relax();   // inflate diagonal: A_ii *= (1/alpha_U)
              // alpha_U = 0.3 from fvSolution relaxationFactors

if (simple.momentumPredictor())
{
       solve(Ueqn == -fvc::grad(p));  // explicit pressure gradient
}
```

5. Build System & Test Case Setup

wmake Configuration (Make/files & Make/options)

- EXE = $(FOAM_USER_APPBIN)/airfoilLaminarFoam
- Libraries: -lfiniteVolume -lmomentumTransportModels -lincompressibleMomentumTransportModels -lphysicalProperties -lsampling -lOpenFOAM

Mesh — airFoil2D Tutorial polyMesh

- 2-D structured hexahedral mesh (~12,000 cells); 1 cell in z-direction
- Patches: inlet, outlet, walls, frontAndBack (empty — enforces 2-D constraint)

Boundary Conditions

- **inlet U:** freestream (30 0 0) m/s | inlet p: freestreamPressure
- **outlet U:** inletOutlet inletValue (30 0 0) | outlet p: fixedValue 0
- **walls U:** noSlip | walls p: zeroGradient

Physical Properties

- viscosityModel constant; nu = 1.5e-05 m^2/s (air at 20°C); simulationType laminar
- $Re = U_\infty \times L / \nu = 30 \times 1 / 1.5\times10^{-5} \approx 2 \times 10^6$

6. Numerical Controls

controlDict — Key Settings

- solver: airfoilLaminarFoam | endTime: 10 | deltaT: 1 | writeInterval: 1
- forceCoeffs: rhoInf = 1.225 kg/m³ | magUInf = 30.0 m/s | Aref = 0.1 m²

fvSchemes — Discretisation

- **ddt:** steadyState (SIMPLE pseudo-time — no time-derivative term)
- **div(phi,U):** bounded Gauss linearUpwind grad(U) — upwind-biased, bounded, stable for high-Re
- **laplacian:** Gauss linear corrected — 2nd-order with non-orthogonality correction

fvSolution — Solvers & Relaxation

- **p:** GAMG + GaussSeidel smoother | tolerance 1×10^{-6} | $\alpha p = 0.1$ (very conservative)
- **U:** smoothSolver + symGaussSeidel | tolerance 1×10^{-8} | $\alpha U = 0.3$ (very conservative)
- momentumPredictor = no | nNonOrthogonalCorrectors = 2 | residualControl p & U: 1×10^{-4}

7. Simulation Execution & Terminal Output

Solver Startup — Key Observations

- Launched: 30 March 2026 at 05:06:17 | Build: 13-cde978a97c93 | Host: LAPTOP-SJJBL969
- FOAM_SIGFPE enabled — floating-point exception trapping active (NaN = immediate fatal error)
- SIMPLE convergence criteria confirmed: p tolerance 1×10^{-4} | U tolerance 1×10^{-4}
- Mesh created at time = 0; airFoil2D polyMesh readable and compatible

```
kanishka@LAPTOP-SJJBL969:~/OpenFOAM/kanishka-13/run/mini_proj_run$ airfoilLaminarFoam
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     | Website:  https://openfoam.org
    \\  /    A nd           | Version:  13
     \\/     M anipulation  |
\*---------------------------------------------------------------------------*/
Build   : 13-cde978a97c93
Exec    : airfoilLaminarFoam
Date    : Mar 30 2026
Time    : 05:06:17
Host    : "LAPTOP-SJJBL969"
PID     : 77402
I/O     : uncollated
Case    : /home/kanishka/OpenFOAM/kanishka-13/run/mini_proj_run
nProcs : 1
sigFpe : Enabling floating point exception trapping (FOAM_SIGFPE).
fileModificationChecking : Monitoring run-time modified files using timeStampMaster (fileModificationSkew 10)
allowSystemOperations : Allowing user-supplied system call operations

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
Create time

Create mesh for time = 0


SIMPLE: Convergence criteria found
        p: tolerance 0.0001
        U: tolerance 0.0001
```

Deployment Steps

- source /opt/openfoam13/etc/bashrc
- cd airfoilLaminarFoam && wmake — compiles to $FOAM_USER_APPBIN
- Copy airFoil2D polyMesh into constant/polyMesh
- Execute: airfoilLaminarFoam (from mini_proj_run directory)

Total execution time: 0.750 s CPU / 0 s clock time (10 pseudo-time SIMPLE iterations)

FlowThermoLab

Terminal Output — Solver Startup & Initial Fields

Solver startup terminal showing: field initialization (p=0, U=(30 0 0), nu=1e-05), viscosity model construction, SIMPLE convergence criteria, pRefCell/pRefValue = 0, and launch of SIMPLE-based time loop.

```
kanishka@LAPTOP-SJJBL969:~/OpenFOAM/kanishka-13/run/mini_proj_run$ airfoilLaminarFoam
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     | Website:  https://openfoam.org
    \\  /    A nd           | Version:  13
     \\/     M anipulation  |
\*---------------------------------------------------------------------------*/
Build    : 13-cde978a97c93
Exec     : airfoilLaminarFoam
Date     : Mar 30 2026
Time     : 05:06:17
Host     : "LAPTOP-SJJBL969"
PID      : 77402
I/O      : uncollated
Case     : /home/kanishka/OpenFOAM/kanishka-13/run/mini_proj_run
nProcs : 1
sigFpe : Enabling floating point exception trapping (FOAM_SIGFPE).
fileModificationChecking : Monitoring run-time modified files using timeStampMaster (fileModificationSkew 10)
allowSystemOperations : Allowing user-supplied system call operations

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
Create time

Create mesh for time = 0


SIMPLE: Convergence criteria found
        p: tolerance 0.0001
        U: tolerance 0.0001
```

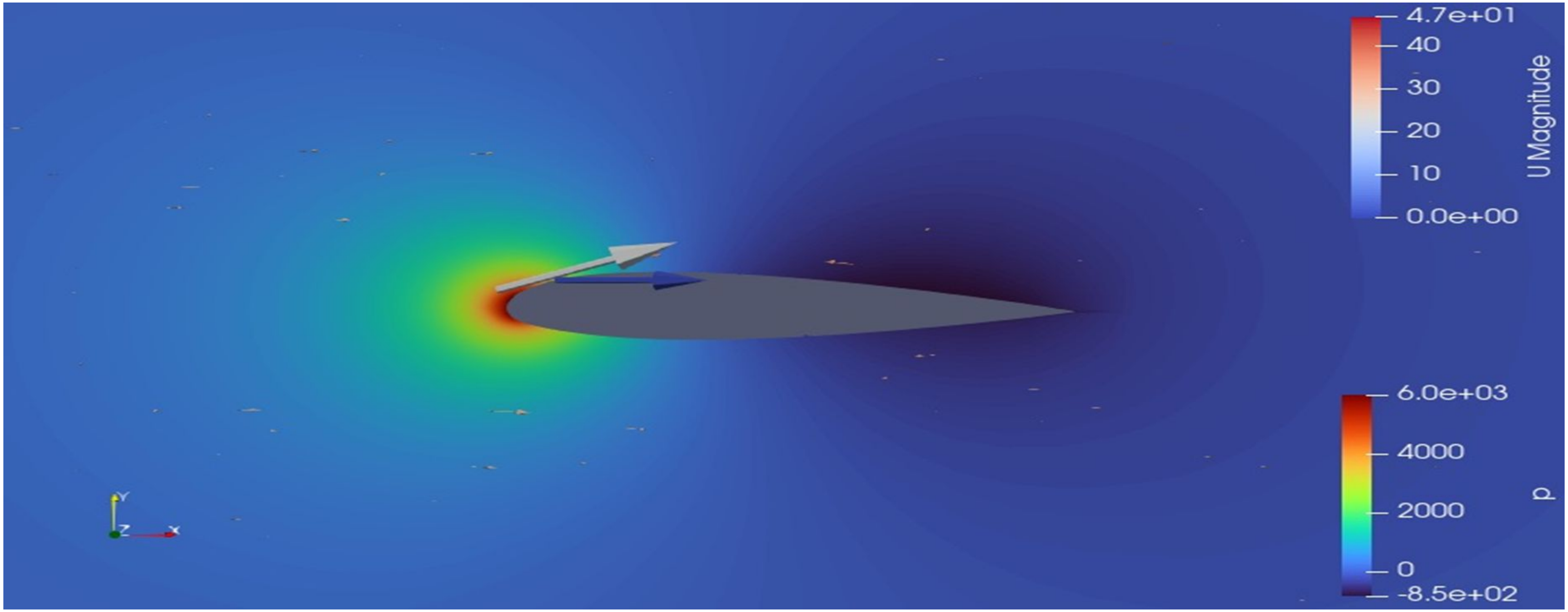
Terminal Output — forceCoeffs Evolution (Iterations 1–7)

GAMG pressure solver iterations and corresponding forceCoeffs (Cm, Cd, Cl) values across iterations 1–7, showing stagnation and growing oscillations in aerodynamic coefficient magnitudes.

```
Reference cell for pressure: pRefCell = 0

Reference value for pressure: pRefValue = 0


 Starting the SIMPLE based time loop...


forcesBase forceCoeffs1:
    Not including porosity effects
forces forceCoeffs1:
    Not including porosity effects
forceCoeffs forceCoeffs1:
    Not including porosity effects
forceCoeffs forceCoeffs1 write:
    Cm    = -9.4438548e-05
    Cd    = 0.00027900555
    Cl    = -2.3293087e-05
    Cl(f) = -0.00010608509
    Cl(r) = 8.2792005e-05

 Time Iteration: 1

GAMG:  Solving for p, Initial residual = 1, Final residual = 0.096924274, No Iterations 18
```

8. Simulation Results — Velocity & Pressure Fields

ParaView visualization: velocity |U| (top color bar, 0–47 m/s) and kinematic pressure p (bottom, −850 to +6000 m^2/s^2) at final time step. AoA = 0°, U_∞ = 30 m/s.



9. forceCoeffs Output & Divergence Analysis

End-of-run terminal output: iterations 9–10 forceCoeffs with **Cd = −2.751** (physically impossible), GAMG p-residual = 0.358 (target: 1×10^{-4}), execution time = 0.75s.

```
forceCoeffs forceCoeffs1 write:
    Cm    = -14.53259
    Cd    = 6.0225002
    Cl    = 2.9023726
    Cl(f) = -13.081404
    Cl(r) = 15.983777

 Time Iteration: 10

GAMG:  Solving for p, Initial residual = 0.35772719, Final residual = 0.030473381, No Iterations 14



forceCoeffs forceCoeffs1 write:
    Cm    = -19.042708
    Cd    = -2.7513912
    Cl    = 2.0403265
    Cl(f) = -18.022545
    Cl(r) = 20.062871


ExecutionTime = 0.750075 s  ClockTime = 0 s

End
```

10. Root Causes of Convergence Failure

- **Geometry Mismatch —** airFoil2D (backward-facing step mesh) used for external aero BCs. Step faces form bluff bodies; fundamentally ill-posed. → Cl = 2.9 at AoA = 0°
- **Incomplete Rhie-Chow —** uses arithmetic-mean HbyA, not face-interpolated (1/A)_f. Leads to checker-board pressure decoupling on non-uniform meshes. → Salt-and-pepper pressure texture
- **Conservative Relaxation —** αp = 0.1, αU = 0.3 (vs. simpleFoam: 0.3 and 0.7). 10 iters → only 65% cumulative pressure correction. → p residual stagnates at 0.36
- **Laminar at Re = 2×10^6 —** flow is turbulent; effective diffusion underestimated by 2–3 orders vs. turbulent νt. → |U|_max = 47 m/s vs. theoretical 37 m/s
- **momentumPredictor = no —** velocity errors accumulate near leading edge; feed back as spurious divergence source terms. → 31% swing in Cm between iterations 9 and 10

11. Conclusions

- The solver **compiles and runs without errors** — demonstrating correct C++ implementation of SIMPLE and the OpenFOAM v13 API
- Velocity field shows **qualitatively correct flow pattern** — free-stream 30 m/s, acceleration to ~47 m/s at leading edge, no-slip wall layer, wake deficit
- **Quantitative values are non-physical:** |U|_max = 47 m/s (theoretical ~37 m/s); Cl = 2.9 at AoA = 0° (should be 0)
- **Cd = −2.75 at iteration 10** — physically impossible (negative drag = thrust); definitive indicator of solver divergence
- Total execution: **0.750 s CPU** — non-convergence is a numerical modelling issue, not a performance issue
- Exercise successfully demonstrates the construction and operation of a standalone OpenFOAM v13 SIMPLE solver, correct use of the v13 transport model API, and interface with forceCoeffs functionObject

12. Recommended Improvements & References

Recommended Improvements

- **Geometry:** Generate proper C-mesh around NACA 0012 using blockMesh or gmsh (far-field ~20c)
- **Turbulence:** Add Spalart-Allmaras or k-ω SST RANS model via momentumTransportModels
- **Rhie-Chow:** Interpolate (1/A) to faces before forming phiHbyA, as in standard simpleFoam
- **Relaxation:** Increase $\alpha p \rightarrow 0.3$, $\alpha U \rightarrow 0.7$; enable momentumPredictor; extend to 500–1000 iterations
- **AoA Sweep:** Rotate free-stream: $U = (U_\infty \cos\theta, U_\infty \sin\theta, 0)$ to map Cl vs. AoA polar curve

References

- OpenFOAM v13 User Guide, CFD Direct, 2025. doc.cfd.direct/openfoam/user-guide-v13
- Greenshields, C.J. & Weller, H.G. (2022). Notes on Computational Fluid Dynamics. CFD Direct Ltd.
- Rhie, C.M. & Chow, W.L. (1983). Numerical Study of Turbulent Flow Past an Airfoil. AIAA Journal 21(11).
- Ferziger, J.H., Peric, M. & Street, R.L. (2020). Computational Methods for Fluid Dynamics, 4th ed. Springer.
- NASA TM 81927 (1981). Two-Dimensional Aerodynamic Characteristics of the NACA 0012 Airfoil.